

智能任务协同看板项目

团队协同研发基本盘手册 (SDD)

项目名称	智能 AI 驱动任务协同看板系统 (基于 Spec Coding 与 Trae 生态)	治理模型	四人制人类研发治理矩阵 (宏观把控)
核心范式	规范驱动开发 (Spec-Driven Development, SDD)	技术底座	Spring Boot 3 + Vue 3 + PostgreSQL
文档版本	v1.0.0 (生产交付级)	发布日期	2026年5月17日

1. 范式转移与全景概述

本手册旨在为团队全体成员（共 4 人）提供统一的数字工程底座。项目彻底放弃传统的串行开发分工，全面转向由 **Trae IDE SOLO自治模式** 驱动的 **Spec Coding（规范驱动编码）** 协作流。研发流程已被重构为“产品设计 Agent”、“研发Agent”与“测试Agent”组成的自动化流水线，人类团队则升级为系统架构的编排者、规则制定者及质量终极审核者。

2. 从 GitHub 克隆至本地的初始化环境配置

为确保全员本地环境与 Agent 运行时完全同频，集成工程师及全体成员必须严格遵循以下步骤执行本地沙箱初始化：

步骤 2.1: 代码仓库克隆与目录切入

打开终端，将 GitHub 远端代码库克隆至本地，并切换至项目根目录：

```
$ git clone https://github.com/wangshun/Dashboard_System.git
$ cd Dashboard_System
```

步骤 2.2: 环境依赖校验

OpenSpec 框架对文件系统 API 有严格的版本依赖，强制要求本地本地 Node.js 版本环境满足以下要求：

```
$ node -v
v20.19.0 (或更高 LTS 版本)
```

步骤 2.3: 全局安装 OpenSpec 命令行工具链

在本地全局安装官方核心编排引擎，确保在任意目录下能够直接调度 `openspec` 指令：

```
$ npm install -g @fission-ai/openspec@latest
$ openspec --version
```

步骤 2.4: 校验项目基础结构

当前项目已经执行过 `openspec init`。请确保本地存在以下底层目录，它们构成了 OpenSpec 制品流转的生命周期核心：

- `openspec/specs/`：长期系统行为规范库（Delta Specs，事实的唯一来源）。
- `openspec/changes/`：日常需求变更提议区（存放 `proposal.md` 和 `tasks.md`）。
- `openspec/archive/`：变更合并后的历史规范审计归档区。

3. 项目技术宪法 (Project Constitution)

在根目录下的 `project.md` 中，系统架构师已确立了以下全局硬约束。任何 Agent 的代码生成与重构均不得偏离本宪法：

- **技术栈锁定**：前端强制使用 `Vue 3` (Composition API) 结合 `Vue Draggable` 搭建看板组件；后端强制使用 `Spring Boot 3` 与 `Spring AI` 模块；持久层使用 `PostgreSQL`。
- **通信与路由红线**：看板拖拽操作必须通过 `WebSocket` 协议维持多客户端的毫秒级实时状态同步；前端路由守卫与后端拦截器强制采用 `JWT` 进行无状态安全认证。
- **数据架构约束**：严禁引入易产生状态不一致的本地缓存，强制依赖 Redis 级外部硬约束进行全局状态管理；任务表结构中必须保留 `JSONB` 灵活数据类型以应对未来扩展需求。

4. 三元智能体 (Agents) 拓扑配置

项目在 `.trae/agents/` 下固化了三大总控智能体配置文件，明确界定了它们的角色边界与挂载技能 (Mounted Skills)：

智能体名称 (配置文件)	角色定位与生命周期	挂载技能矩阵 (Mounted Skills)	核心行为红线
产品设计 Agent ProductDesignAgent.md	位于研发最前端，主要在 Trae 的 IDE 交互模式下运作。	EARS_Requirement Prototype openspec-propose openspec-explore	绝对禁止写代码。 在提议阶段只允许输出规范和清单 Markdown。
研发 Agent RnDAgent.md	系统全面建设者，主要在 Trae 的 SOLO 自主模式下独立运行。	PostgreSQL_Design PortDesign SpringVue_Coding openspec-apply-change	必须严格按 tasks.md 的序列单步执行并打钩，严禁跳跃式重构。
测试 Agent TestingAgent.md	质量守门员，对抗性实体，同样在 Trae SOLO 模式下被激活。	testcase-xmind-generator api-doc-to-pytest web-ui-playwright-mcp bug-report-generator	测试失败必须触发根本原因追踪日志并打回，全绿方可放行。

5. 八大核心能力模块 (Skills) 资产清单

技能存放在 `.trae/skills/` 目录下，作为控制 Agent 行为准则的细粒度说明书：

5.1 产品设计域 Skill

- **EARS_Requirement (需求规范Skill)**：强制要求 Agent 将自然语言转化为完全无歧义的 EARS 语法，消灭模糊字眼。
- **Prototype (原型Skill)**：通过指令或代码描述看板界面的视觉隐喻，基于 Tailwind CSS 规范或 Vue 组件树确立任务详情侧边栏、看板列形态。

5.2 研发域 Skill

- **PostgreSQL_Design (表设计Skill)**：驱动 Agent 自动规划符合第三范式的高效建表脚本，强制包含审计字段（`created_at` 等）并优化 `JSONB` 字段设计。
- **PortDesign (接口设计Skill)**：规范数据交换契约，要求 HTTP API 必须遵循严格的 RESTful 规范，定义清晰的请求响应体，并针对看板拖拽交互规划双向 WebSocket 消息载荷（Payload）协议。
- **SpringVue_Coding (编码Skill)**：研发 Agent 编码引擎。受到技术栈（Spring Boot 3 + Vue 3）红线绝对约束，确保 SOLO 模式安全调用物理工具。

5.3 测试与流程域 Skill

- **TestCaseDesignSkill** / **api-doc-to-pytest** (用例设计Skill): 应用边界值分析和等价类划分, 从 EARS 规范中逆向生成覆盖正向路径与负向异常分支的结构化测试矩阵。
- **web-ui-playwright-mcp** / **IntegrationTestSkill** (自动化测试Skill): 生成单元测试及端到端 (E2E) 脚本, 调度无头浏览器模拟光标移动, 验证卡片拖拽状态同步。
- **openspec-*** 系列 (规范层控制Skill): 涵盖 **propose**, **apply**, **archive**, **explore** 的控制流, 确保生命周期的阶段性流转。

6. 模型上下文协议 (MCP) 物理连接矩阵

项目级 MCP 配置文件固化于 `.agents/mcp.json` 中, 为 Agent 赋予了操作真实世界的物理抓手:

- **project-filesystem** (文件系统 MCP): 限制 Agent 只能在当前项目根目录的沙箱范围内进行 Vue 和 Java 源码的读写遍历, 严禁越界访问。
- **github-mcp** (GitHub 通道): 使 Agent 具备自动管理代码分支、提交 Pull Request 的能力, 便于团队进行异步代码审查。
- **figma-mcp** (设计联动): 使产品设计 Agent 能够读取团队 Figma 原型库中的视觉隐喻与设计变量。
- **playwright-mcp** (测试浏览器): 提供无头浏览器的运行控制权, 模拟人类用户交互。
- **postgres-mcp** (关系型数据库连接): 允许测试 Agent 在测试沙箱内执行 DDL 建表或在 Dnd 测试前后自动执行数据清理脚本。

⚠ 团队协作安全红线 (最高优先级):

`mcp.json` 中配置的 `${GITHUB_TOKEN}`、`${FIGMA_TOKEN}` 与 `${DATABASE_URL}` 属于本地动态插值变量。严禁将任何含有明文私钥、密码的代码 commit 提交至 GitHub 仓库! 四名人类团队成员必须在各自电脑的系统环境变量或 Trae 的独立环境配置中声明上述密钥, 确保开箱即用且安全隔离。

7. 规范驱动开发 (SDD) 全生命周期流转路径

团队在日常迭代一个新功能 (如“实时看板卡片拖拽同步”或“AI智能推荐优先级”) 时, 必须严格执行以下链式调用生命周期:

1. 提议阶段 (Propose Phase)

领域专家在 Trae IDE 交互窗口发起高阶指令, 触发 `/opsx: propose`。产品设计 Agent 被唤醒, 调用 `EARS_Requirement` 与 `Prototype` 技能, 在 `changes/` 下生成 `proposal.md` 阐明变更动机。

2. 规范与技术设计阶段 (Specify & Design Phase)

产品设计 Agent 采用 **EARS 语法** 编写 `specs/*.md` (Delta Specs) , 例如:

"When 用户将任务卡片拖拽至不同的看板列时, 系统 **shall** 通过 WebSocket 触发多端状态更新, 并将任务状态变更为目标列状态。"

随后调用表设计和端口设计 Skill 输出 `design.md` 与包含原子实现检查项的 `tasks.md`。系统架构师执行人工在环审核, 确认表结构与接口设计合理。

3. 自动化代码执行阶段 (Apply Phase)

人类执行 `/opsx: apply` 快捷键一键切入 **Trae SOLO 自主运行模式**。研发 Agent 进驻沙箱, 以 `tasks.md` 为唯一绝对行动指南, 通过文件系统 MCP 编写或修改 Vue 3 拖拽视图与 Spring Boot 业务代码。每实现一步, Agent 必须主动将清单改为 `-[x]` 以同步进度, 绝对禁止跳步或自由重构。

4. 对抗性测试与自愈修复阶段 (Test & Self-Healing Phase)

测试 Agent 被激活, 逆向提炼测试矩阵, 并调度 Playwright MCP 移动光标执行 E2E 看板状态同步冲突测试。若发生断言失败, 捕获精确的堆栈日志反馈给研发 Agent, 自动启动闭环自愈修复, 直至全绿通过。

5. 合规归档阶段 (Archive Phase)

Eval 审计工程师提取并审核测试报告, 在终端执行 `/opsx: archive`。此次开发的增量规范被正式合入项目长期规范库 `specs/` 中, 代码并入主干, 本次变更包移入 `archive/` 区以备软著申请或课题审计。

8. 人类治理矩阵、AI 日志标准化与注意事项

8.1 四人人类团队 RACI 治理矩阵

- **系统架构师 / Agent设计师**: 主导初始化与架构决策, 负责 `project.md` 技术宪法的编撰与调优, 管理 AI 提示词上下文, 对设计方案最终把关。
- **集成 / 粘合工程师**: 负责维护 MCP 服务器环境、管理密钥安全、搭建数据库实例, 保障四人本地开发环境的高度一致性。
- **评估 / 审计工程师**: 专注于质量护栏, 审查测试 Agent 输出的报告, 分析大模型推理中的逻辑幻觉, 执行红蓝对抗。
- **领域专家 / 业务仲裁者**: 代表最终用户, 在 IDE 下发起高阶自然语言需求, 行使决策链条中的“人工在环”一键否决权, 对 proposal 进行最终成果验收。

8.2 AI 辅助日志 (AI Logs) 标准化要求

为防止自治 SOLO 系统产生黑盒效应，所有 Agent 在推理时产生的辅助日志必须遵循统一标准，将瞬时思维模型解构为强制性的 **DID / INSIGHT / NEXT-STEP**（做了什么 / 认知洞察 / 下一步行动）代码块结构，并记录具体的 **chat_id**、所调用的 MCP 函数（如 **execute_sql_migration**）以及消耗的 Token 数量，以实现推理决策因果链条的绝对可审计性。

8.3 协作避坑红线摘要

1. **同频同步**：任何关于 **.trae/** 目录内 Skill 提示词或约束规则的调整，修改人必须及时 Push 远端。其余三名 Human 成员必须执行 **git pull** 并重启 Trae 以更新 AI 大脑，严禁本地私造规则。
2. **单步执行护栏**：研发 Agent 在 SOLO 模式下编码时，如发现其产生“自由发散”或连续跳步现象，集成工程师必须立即阻断流转，将其踢回上一任务节点，防止上下文污染。